Non-Primitive Data Type or Reference Data Types 1
Single-Dimensional Arrays 1
Demonstrate Array 4
Strings 5
Demonstrate Strings 8

Non-Primitive Data Type or Reference Data Types

The Reference Data Types will contain a memory address of variable values because the reference types won't store the variable value directly in memory. They are strings, objects, arrays, etc.

Primitive data type vs. non-primitive type in Java

Primitive Datatypes	Non-Primitive Datatypes
Primitive Datatypes are pre-defined by the language itself.	Non-Primitive Datatypes need to be defined by the user.
When declared primitive data is stored in the stack.	The reference variables are stored in the stack but the original object is in the heap.
When a copy is created for the data, a completely separate allocation is done for the copy.	Copies here create new reference variables but they both point to the same object in the heap.
Thus, the changes made to the copy are not reflected in the original.	Here however the changes made to the copy trace back to the original.
The primitive data structure will contain some value, i.e., it cannot be NULL.	The non-primitive data structure can consist of a NULL value.
The size depends on the type of data structure.	In the case of a non-primitive data structure, size is not fixed.
The primitive data structure can be used to call the methods.	Non-primitive data structure cannot be used to call the methods.

Single-Dimensional Arrays

Lesson link: https://www.w3schools.com/java/java_arrays.asp

- An **array** is an ordered, fixed-size container that allows you to store multiple values of the **same data type** inside a single variable name.
- Instead of creating ten separate variables to hold ten student test scores (score1, score2, etc.), you create one single array variable named scores that holds all ten values together in a neat list.

1. Visualizing Arrays and Indexes

Like strings, arrays use a zero-based numbering system to track where items live. Each slot in the array is called an **element**, and its coordinate position is called its **index**.

Let's look at an integer array containing four numbers: `[10, 20, 30, 40]`

text

```
 Element Value: |  10  |  20  |  30  |  40  |
                |------|------|------|------|
  Index Number: |   0  |   1  |   2  |   3  |
```

Use code with caution.

- **Array Length:** 4
- **First Element:** Index 0 (`10`)
- **Last Element:** Index 3 (`40`) — *Always the Length minus 1!*

2. Creating Arrays in Java

There are two distinct ways to declare and initialize an array in Java:

Method A: The Literal Way (When you already know the values)

Use curly braces `{}` to plug data straight in when creating the variable.

java

```java
int[] luckyNumbers = {7, 13, 21, 42};
String[] fruits = {"Apple", "Banana", "Orange"};
```

Use code with caution.

Method B: The Blank Blueprint Way (When you don't know the values yet)

Use the `new` keyword and declare an absolute, fixed size. Java will build an empty list filled with default values (like zeros). **You cannot resize it later.**

java

```java
// Builds an empty list with room for exactly 5 integers
int[] studentGrades = new int[5]; // Stores: [0, 0, 0, 0, 0]
```

3. Reading and Writing Array Values

To look inside a slot or change a value, use square brackets `[]` paired with the target index number.

java

```java
int[] scores = {90, 85, 77};

// 1. READ a value from index 1
System.out.println(scores[1]); // Prints: 85

// 2. WRITE/UPDATE a value at index 2
scores[2] = 95; // Changes 77 to 95

// Array is now: [90, 85, 95]
```

4. Processing Arrays with Loops

Because arrays rely on sequential numbers (`0, 1, 2, 3...`), they are perfect partners for **for loops**. The loop counter acts as the index pointer to scan through the elements automatically.

Standard For Loop (Best for modifying or tracking indexes)

Every array has a built-in `.length` attribute that tells you how many items are inside.

java

```java
int[] runTimes = {12, 15, 11, 14};

for (int i = 0; i < runTimes.length; i++) {
    System.out.println("Runner " + i + " finished in " + runTimes[i] + " mins");
}
```

Enhanced For Loop / "For-Each" (Best for quickly reading data)

If you only need to look at every value without tracking index coordinates, use this clean syntax. It reads aloud as: *"For every individual `time` inside the `runTimes` list..."*

java

```java
int[] runTimes = {12, 15, 11, 14};

for (int time : runTimes) {
    System.out.println("Time: " + time);
}
```

Demonstrate Array

Example demonstrating how to **declare, initialize, and loop through** an array in Java.

```
public class ArrayDemo {
    public static void main(String[] args) {
        // 1. Declare and initialize an array of Strings with a size of 4
        String[] fruits = {"Apple", "Banana", "Cherry", "Mango"};

        // 2. Access elements using an index (Note: Java arrays start at index 0)
        System.out.println("First fruit: " + fruits[0]); // Outputs: Apple
        System.out.println("Third fruit: " + fruits[2]); // Outputs: Cherry

        // 3. Modify an existing element
        fruits[1] = "Blueberry"; // Changes "Banana" to "Blueberry"

        // 4. Get the size of the array
        System.out.println("\nTotal items in array: " + fruits.length);

        // 5. Loop through the array using an enhanced 'for-each' loop
        System.out.println("--- Current Fruit List ---");
        for (String fruit : fruits) {
            System.out.println("- " + fruit);
        }
    }
}
```

Expected Output

```
First fruit: Apple
Third fruit: Cherry

Total items in array: 4
--- Current Fruit List ---
- Apple
- Blueberry
- Cherry
```

Strings

Lesson link
https://www.w3schools.com/java/java_strings.asp

- Strings are defined as an array of characters / sequence of characters that is held in single variable. Used to represent text (like names, passwords, or sentences).
- Character array (not String) is a collection of separate char-type entities accessed by indexing.
- Unlike C/C++, Java strings are not terminated with null characters.
- **Syntax:** Declaring a string
  - <String_Type> <string_variable> = “<sequence_of_string>”;
  - String s1 = "Hello"; // Stored in String Pool
  - String s2 = "Hello"; // Reuses the same object from the String Pool
  - String s3 = new String("Hello"); // Creates a new String object outside the Pool
- The Key Rule: Strings are Immutable

  That is once String is created, **it can never be changed in memory**.
  String manipulation methods that "modify" text, Java doesn't alter the original word. Instead, it quietly creates a brand-new String in the background and hands you fresh copy.
- Strings is object and provides built-in function (methods) to search, cut, clean, and modify text.
- Built-in String operation functions are, such as finding length, extracting substrings, replacing characters, and more.

Operation	Method
Get Length	length()
Get Character	charAt(int index)
Extract Substring	substring(int start, int end)
Find Substring	indexOf(String substr)
Replace Characters	replace(char old, char new)
Change Case	toUpperCase(), toLowerCase()
Split String	split(String regex)

- Built-in String comparing functions
  Golden Rule of Comparison, use equals() Instead of ==. The == operator compares references, not the actual content of the strings, whereas equals() compares the values.

Method	Purpose
equals()	Compares content
equalsIgnoreCase()	Ignores case differences
compareTo()	Lexicographical comparison

- Substring functions
  Extracts a smaller piece of text out of a larger String. It takes a starting index and an optional ending index. *Note:* The ending index is **exclusive** (Java stops right *before* that number).

2. Visualizing String Indexes

To manipulate text, you must understand how Java views a String. Java assigns a number position (called an **index**) to every single character, **starting exactly at 0**. Spaces count as characters too!

Let's look at the index mapping for the text `"Java App"`:

text

```
Character: |  J  |  a  |  v  |  a  |     |  A  |  p  |  p  |
           |-----|-----|-----|-----|-----|-----|-----|-----|
     Index: |  0  |  1  |  2  |  3  |  4  |  5  |  6  |  7  |
```

Use code with caution.

- **Total Characters (Length):** 8
- **First Character Index:** 0 (`'J'`)
- **Last Character Index:** 7 (`'p'`) — *Always Length minus 1!*

```
String phrase = "Superstar";

// Cut from index 5 all the way to the end String
endPiece = phrase.substring(5); // "star"

 // Cut from index 0 up to (but not including) index 5
String frontPiece = phrase.substring(0, 5); // "Super"
```

- Advance methods

```
String.format() allows you to create formatted strings dynamically.
String formatted = String.format("Name: %s, Age: %d", "Alice", 30);
System.out.println(formatted); // "Name: Alice, Age: 30"

String joined = String.join("-", "2023", "12", "31"); // "2023-12-31"

String s = "  Hello  ";
System.out.println(s.strip());  // "Hello" Removes leading and trailing whitespace.
System.out.println("   ".isBlank());  // true
System.out.println("Hi".repeat(3));  // "HiHiHi"

Regular Expressions & Pattern Matching
String email = "user@example.com";
boolean isValid = email.matches("^[\\w.-]+@[\\w.-]+\\.[a-zA-Z]{2,6}$");
```

- Built-in Java methods used to modify

```
String CSVData = "Pizza,Burgers,Tacos";
String[] foodList = CSVData.split(",");

// foodList[0] is now "Pizza"
// foodList[1] is now "Burgers"
// foodList[2] is now "Tacos"
```

Method Signature	What It Does (Action)	Input Example	Output Result
`.trim()`	Removes all blank spaces from the very beginning and very end.	`" Hello World ".trim()`	`"Hello World"`
`.toLowerCase()`	Converts every letter to lowercase.	`"Java123".toLowerCase()`	`"java123"`
`.toUpperCase()`	Converts every letter to uppercase.	`"code".toUpperCase()`	`"CODE"`
`.replace(old, new)`	Swaps out a target character or word with a new one.	`"apple".replace('p', 'b')`	`"abble"`
`.substring(start)`	Cuts text starting from the chosen index to the very end.	`"Superstar".substring(5)`	`"star"`
`.substring(start, end)`	Cuts text from the `start` index **up to but not including** the `end` index.	`"Superstar".substring(0, 5)`	`"Super"`
`.concat(str)`	Glues another string to the end of the current string.	`"Ice".concat("Cream")`	`"IceCream"`

Demonstrate Strings

Example demonstrating how to use **Java Strings**, showcasing common operations like finding length, changing case, checking contents, and joining text.

```
public class StringDemo {
   public static void main(String[] args) {
      // 1. Declare and initialize a String
      String text = "  Hello, Java Learners!  ";
      // 2. Get the length of the string
      System.out.println("Original length: " + text.length());
      // 3. Remove leading and trailing whitespace using trim()
      String cleanText = text.trim();
      System.out.println("Cleaned text: '" + cleanText + "'");
      // 4. Convert case (Uppercase / Lowercase)
      System.out.println("Uppercase: " + cleanText.toUpperCase());

      // 5. Extract a portion of the text using substring(startIndex, endIndex)
      // Extracts characters from index 7 up to (but not including) index 11
      String word = cleanText.substring(7, 11);
      System.out.println("Extracted word: " + word); // Outputs: Java

      // 6. Check if the string contains a specific word
      boolean hasJava = cleanText.contains("Java");
      System.out.println("Contains 'Java'?: " + hasJava);

      // 7. Replace characters or words
      String newText = cleanText.replace("Learners", "Experts");
      System.out.println("Replaced text: " + newText);

     String s1 = new String("Apple");
     String s2 = new String("Apple");
    System.out.println(s1 == s2);     // ❌ Prints false (different memory objects)
    System.out.println(s1.equals(s2)); //  Prints true (same characters)
   }
}
```

Expected Output

```
Original length: 25
Cleaned text: 'Hello, Java Learners!'
Uppercase: HELLO, JAVA LEARNERS!
Extracted word: Java
Contains 'Java'?: true
Replaced text: Hello, Java Experts!
```

Crucial Tip for Beginners: Comparing Strings

In Java, **never use ==** to compare the text content of two Strings. Always use the **.equals()** method.

- == checks if the strings are the exact same **object in memory**.
- .equals() checks if the **characters inside** the strings match.